

Rapport Labo 2

Kenan Augsburg

08 November 2025

Contents

1. Code analysis	3
1.1. Global constants	3
2. Errors	4
2.1. <code>random.randrange</code>	4
2.1.1. Bad range $\{0, \dots, N - 1\}$	5
2.1.2. Usage of <code>random.randrange</code>	6
2.2. <code>load_or_generate_keys</code>	6
2.2.1. Blind loading	6
2.2.2. Destructive storage of private key	11
2.2.3. Insecure permission management of private key	11
2.3. Domain issues	12
3. Conclusion	18

1. Code analysis

The code analysis is done in the code in comments. If the code has been modified, the original comments can be found in the section addressing the issues or in some cases, the function is suffixed with `_old` and decorated with `@deprecated`

1.1. Global constants

At the start of the scripts, we have multiple constants defined which are the parameters for the secp256k1

```
1 # --- secp256k1 parameters ---
2 P = 0xFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFEFFFFC2F
3 A = 0
4 B = 7
5 Gx
  = 55066263022277343669578718895168534326250603453777594175500187360389116729240
6 Gy
  = 32670510020758816978083085130507043184471273380659243275938904335757337482424
7 N = 0xFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFEBAAEDCE6AF48A03BBFD25E8CD0364141
```

En comparant avec les paramètres que l'on trouve en ligne¹ on peut confirmer que ces paramètres sont correctes.

¹<https://neuromancer.sk/std/secg/secp256k1>

2. Errors

☰ Tâche 1

For each error:

- Show the snippet
- Explain what the problem is and the implications. What an attacker can do and what could go wrong.
- Explain in simple terms why this should be fixed
- Give a fix

2.1. `random.randrange`

This function is used twice in the code.

```
1 def generate_keypair() → tuple[int, tuple[int, int]]:
2     """Generate a pair of keys"""
3     priv = random.randrange(1, N)
4     pub = scalar_mult(priv, (Gx, Gy))
5     return priv, pub
```

Python

```
1 def sign_message(priv, message):
2     z = sha256(message)
3     while True:
4         k = random.randrange(0, N)
5         R = scalar_mult(k, (Gx, Gy))
6         r = R[0] % N
7         if r == 0:
8             continue
9         k_inv = inv_mod(k, N)
10        s = (k_inv * (z + r * priv)) % N
11        if s == 0:
12            continue
13        return (r, s)
```

The documentation² for the module and its function starts with a warning to not use it for security and cryptographic purposes. This kind of warning shouldn't be ignored since it means that the prngs it provides are either not proved to be cryptographically secure or worse, proved to be attackable.

Furthermore, the range can be a problem independently of how good the distribution is. When multiplying a point with a scalar value, if the value is equal to zero or a multiple of N the result will be $\mathcal{O}$ which can be problematic.

For the first usage, the range gives us a value $\text{priv} \in \{1, \dots, N - 1\}$. This means that we shouldn't get a `None` but just to be safe we should have an assertion before returning our keys.

²<https://docs.python.org/3/library/random.html>

For the second usage, we have $k \in \{0, \dots, N - 1\}$ which means that R can be `None` which will raise an exception when accessing `R[0]`. This means that there is a $\frac{1}{N}$ chance of having an issue when trying to sign a message.

We have two problems here.

2.1.1. Bad range $\{0, \dots, N - 1\}$

The problem here is that there is a chance of raising an exception and the implications vary depending on how exceptions are handled by the caller as well as which code paths lead to this function being executed.

The caller shouldn't expect `sign_message` to fail when given valid parameters. Depending on how they handle exceptions, the program could end up revealing informations which could be exploited or end up in an unrecoverable state.

If an attacker can trigger this function a lot, it could be used to leak secrets or for a denial of service attack.

If the exception results in an unrecoverable state such as the program exiting There is a serious risk of service degradation proportional to the usage.

In simple terms, the risk is that the service becomes unavailable or worse, that secrets could be leaked and used by bad actors to falsify prescriptions.

The simple fix is to change the range to $\{1, \dots, N - 1\}$. (Using an assertion as a sanity check and also allows the typing system to narrow after the multiplication)

```
1 def sign_message(priv, message):
2     z = sha256(message)
3     while True:
4         k = random.randrange(1, N)
5         R = scalar_mult(k, (Gx, Gy))
6         assert R
7         r = R[0] % N
8         if r == 0:
9             continue
10        k_inv = inv_mod(k, N)
11        s = (k_inv * (z + r * priv)) % N
12        if s == 0:
13            continue
14        return (r, s)
```

As stated previously, the range used in `generate_keypair` is correct but we should still have an assertion after the multiplication since it doesn't cost us much and having $\mathcal{O}$ as a public key would be catastrophic. (since $\forall k \in \mathbb{Z}, k\mathcal{O} = \mathcal{O}$)

```
1 def generate_keypair() → tuple[int, Point]:
2     priv = random.randrange(1, N)
3     pub = scalar_mult(priv, (Gx, Gy))
```

```
4     assert pub
5     return priv, pub
```

2.1.2. Usage of `random.randrange`

📄 Tâche 2

snippet

📄 Tâche 3

implications

📄 Tâche 4

attacker

📄 Tâche 5

what could go wrong

📄 Tâche 6

simple terms why fix

📄 Tâche 7

fix

2.2. `load_or_generate_keys`

2.2.1. Blind loading

```
1  if os.path.exists(priv_file) and os.path.exists(pub_file):
2      # If both files exist, proceed to extract the private and public keys.
3      with open(priv_file, "r") as f:
4          # The private key is converted from a base16 string to an int.
5          # NOTE: The fact that we strip what we read shouldn't matter unless
6          # the key isn't stored properly
7          priv = int(f.read().strip(), 16)
8      with open(pub_file, "r") as f:
9          # The public key is read as a JSON object with the keys x and y
10         # containing integers in base64
11         pub_data = json.load(f)
```

```

12     pub = (int(pub_data["x"], 16), int(pub_data["y"], 16))
13     print("Loaded existing keypair from disk.")
14     # WARN: if both files exist, their value is taken as ground truth. There
15     # is no verification that the private key is in [1;N] and that the
16     # public key is the result of priv * G

```

When loading the pair of keys from disk, there are no checks made to validate it. The problem is that to be able to properly sign and verify the signature of a message you need the private key a to respect $1 \leq a < N$ and the public key A should be $A = aG$.

If $a = 0$, the corresponding public key $A = \mathcal{O}$ which would be unusable and raise an exception whenever accessing either the x or y components of the point.

If $a = N$, then $A = \mathcal{O}$ since N is the order of the group. Same consequences.

Any other value of a will be treated as $a \bmod N$ so as long as $a \bmod N \neq 0$ the key should be valid but there shouldn't be any reason why we should have longer keys since it won't provide any benefits.

If $A \neq aG$, the signature verification won't work.

If A isn't on the curve, the program will stop after an assertion in `scalar_mult`.

If an attacker can modify the keys then you've got bigger problems than the program crashing from invalid keys.

One way this could go wrong is if there is a partial recovery, or maybe one of the keys was missing and the remaining key wasn't writable during the creation of a new pair. In those instances, the pair wouldn't match and lead to the program crashing.

This should be fixed because an invalid pair of keys will make the program unusable.

The simple fix would be to check that the keys are valid right after loading them and raise an exception to be handled otherwise. To make it easier to handle down the line, it also makes sense to split `load_or_generate_keys` into two distinct functions

```

1  def compute_public_key(priv: PrivKey) → PubKey:
2      """Compute the public key of a private key"""
3      pub = scalar_mult(priv, (Gx, Gy))
4      assert pub is not None
5      return pub
6
7
8  def is_valid_private_key(key: PrivKey) → bool:
9      """Check if a private key is valid with the curve"""
10     return 1 ≤ key < N
11
12
13  def is_valid_public_key(pub: PubKey, priv: PrivKey | None) → bool:
14     """Check if a public key is valid

```

```

15     If None is specified for the private key, this function only checks
16     if the
17     """
18     return is_on_curve(pub) if priv is None else pub ==
19     compute_public_key(priv)
20
21     class InvalidPrivateKeyError(Exception):
22         pass
23
24
25     class InvalidPublicKeyError(Exception):
26         pass
27
28
29     def load_priv_key(path: str) → PrivKey:
30         """Load a private key from disk and check its validity
31         Raises:
32             FileNotFoundError: If the path doesn't lead to an existing file
33             InvalidPrivateKeyError: If the key is invalid
34         """
35         if not os.path.exists(path):
36             raise FileNotFoundError()
37         with open(path, "r") as f:
38             priv = int(f.read().strip(), 16)
39             if not is_valid_private_key(priv):
40                 raise InvalidPrivateKeyError()
41             return priv
42
43
44     def load_pub_key(path: str, priv_key: PrivKey | None) → PubKey:
45         """Load a public key from the disk and check its validity.
46         Raises:
47             FileNotFoundError: If the path doesn't lead to an existing file
48             InvalidPublicKeyError: If the public key isn't the public key of the
49             provided private key. If None is given for the private key,
50             The only
51             check is whether the public key is a point on the curve.
52         """
53         if not os.path.exists(path):
54             raise FileNotFoundError()
55         with open(path, "r") as f:
56             # The public key is read as a JSON object with the keys x and y
57             # containing integers in base64
58             pub_data = json.load(f)
59             pub = (int(pub_data["x"], 16), int(pub_data["y"], 16))
60             if not is_valid_public_key(pub, priv_key):

```

```

60         raise InvalidPublicKeyError()
61     return pub
62
63
64 def generate_and_save_keys(priv_path: str, pub_path: str) → tuple[PrivKey,
    PubKey]:
65     if os.path.exists(priv_path):
66         # NOTE: only keeping one backup since I don't want to bother with
        # numbering
67         os.replace(priv_path, priv_path + ".bak")
68         print(
69             f"existing private key found during generation of keys, backing
        up to {priv_path}.bak"
70         )
71
72     if os.path.exists(pub_path):
73         # NOTE: only keeping one backup since I don't want to bother with
        # numbering
74         os.replace(pub_path, pub_path + ".bak")
75         print(
76             f"existing public key found during generation of keys, backing up
        to {pub_path}.bak"
77         )
78
79     priv, pub = generate_keypair()
80     # WARN: The public and private keys are both simply saved to disk
81     # without any specifications when it comes to permissions. (on linux,
82     # resulted in 644 permissions)
83     with open(priv_path, "w") as f:
84         # The private key is simply written properly in hex format
85         f.write(hex(priv))
86     with open(pub_path, "w") as f:
87         # The public key is saved correctly in JSON
88         json.dump({"x": hex(pub[0]), "y": hex(pub[1])}, f)
89     print("New keypair generated and saved.")
90     return (priv, pub)
91
92
93 def generate_and_save_public_key(priv: PrivKey, pub_path: str) → PubKey:
94     if os.path.exists(pub_path):
95         # NOTE: only keeping one backup since I don't want to bother with
        # numbering
96         os.replace(pub_path, pub_path + ".bak")
97         print(
98             f"existing public key found during generation of keys, backing up
        to {pub_path}.bak"
99         )
100

```

```

101     pub = compute_public_key(priv)
102     with open(pub_path, "w") as f:
103         # The public key is saved correctly in JSON
104         json.dump({"x": hex(pub[0]), "y": hex(pub[1])}, f)
105     return pub
106
107
108 def load_or_generate_keys() → tuple[PrivKey, PubKey]:
109     """This function loads an existing pair of private/public keys or
110     generates,
111     saves and return a new pair.
112     """
113     # Files containing the private and public keys
114     priv_file = "ecdsa_private.key"
115     pub_file = "ecdsa_public.key"
116
117     try:
118         priv = load_priv_key(priv_file)
119     except (InvalidPrivateKeyError, FileNotFoundError) as err:
120         print(
121             "Your private key is invalid."
122             if isinstance(err, InvalidPrivateKeyError)
123             else "Missing private key"
124         )
125         choice = input(
126             "Would you like to generate a new pair of keys? Existing keys
127             will be backed up to [key].bak y/N: "
128         )
129         if not choice == "y":
130             raise err
131         return generate_and_save_keys(priv_file, pub_file)
132
133     try:
134         pub = load_pub_key(pub_file, priv)
135     except (InvalidPublicKeyError, FileNotFoundError) as err:
136         print(
137             "Your public key is invalid."
138             if isinstance(err, InvalidPublicKeyError)
139             else "Missing public key"
140         )
141         choice = input("Would you like to generate the public key? y/N: ")
142         if not choice == "y":
143             raise err
144         return (priv, generate_and_save_public_key(priv, pub_file))
145     return (priv, pub)

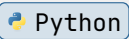
```

Note

This rewrite of the `load_or_generate_keys` is a bit messy but it handles the issue of loading invalid keys as well as the issue of irreversibly losing keys without warning. (Described in more details in the next section)

2.2.2. Destructive storage of private key

```
1 else:
2     print("No keypair found – generating new one...")
3     # If any of the files do not exist we generate a new pair of keys
4     priv, pub = generate_keypair()
5     with open(priv_file, "w") as f:
6         # The private key is simply written properly in hex format
7         f.write(hex(priv))
```



If only private key exists, the function will overwrite it with a new private key. This is destructive since we can always compute the public key from the private key but we cannot recover the private key once overwritten.

This means that if for some reasons, the public key isn't found we will get a new pair of keys. We will need to distribute our new public key to anyone who needs to verify our signatures and if we can't our old public key anywhere, the validity of past records becomes unverifiable.

This most likely isn't an attack vector. If an attacker manages to delete the public key on the server they probably already compromised the server.

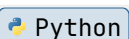
This should be fixed because it's a threat to the trust of previously signed records.

The simple fix to this issue is to prompt the user and ask whether they want to overwrite the private key or compute the public key and save it.

This is implemented in the fix of the previous section.

2.2.3. Insecure permission management of private key

```
1 else:
2     print("No keypair found – generating new one...")
3     # If any of the files do not exist we generate a new pair of keys
4     priv, pub = generate_keypair()
5     with open(priv_file, "w") as f:
6         # The private key is simply written properly in hex format
7         f.write(hex(priv))
8     with open(pub_file, "w") as f:
9         # The public key is saved correctly in JSON
10        json.dump({"x": hex(pub[0]), "y": hex(pub[1])}, f)
11    print("New keypair generated and saved.")
12    # WARN: The public and private keys are both simply saved to disk
13    # without any specifications when it comes to permissions. (on linux,
```



```
14 # resulted in 644 permissions)
```

When either a public key or private key file is missing, a new pair of keys is generated and saved to disk using the built-in open function in 'w' mode.

The keys will have the default permissions (determined by the OS). On Linux, it often means 0644 which is an issue since we don't want any other user to be able to read the private key.

In the event of an attacker gaining access to the host running the program, they can extract the private key without any privilege escalation. This would allow them to falsify records. (Breaks authenticity and integrity)

Effectively, this would mean that the attacker could create a fake prescription and the pharmacy would treat it as valid. For doctors, this would allow the attacker to create fake symptoms for a patient which in turn would result in more work for the doctors to check and effectively destroy the efficiency brought by the system.

This issue should be fixed because it could cause serious legal troubles since it involves medical prescriptions. Furthermore, it also risks the loss of trust in the system.

```
1 priv, pub = generate_keypair()
2 (priv_mask, pub_mask) = (0o600, 0o644)
3 flags = os.O_WRONLY | os.O_CREAT | os.O_TRUNC
4
5 # Get open the file through the os api to enforce permissions on the files
6 priv_fd = os.open(priv_path, flags, priv_mask)
7 os.fchmod(priv_fd, priv_mask)
8
9 with os.fdopen(priv_fd, "w") as f:
10     # The private key is simply written properly in hex format
11     f.write(hex(priv))
12
13 pub_fd = os.open(pub_path, flags, pub_mask)
14 os.fchmod(pub_fd, pub_mask)
15 with os.fdopen(pub_fd, "w") as f:
16     # The public key is saved correctly in JSON
17     json.dump({"x": hex(pub[0]), "y": hex(pub[1])}, f)
18 print("New keypair generated and saved.")
```

2.3. Domain issues

This issue involves the design of the program and isn't specific to a snippet or two of code. This is mainly about what gets signed, how it's structured and which keys are used.

```
1 def build_doctor_record() → bytes:
2     """Build a json array with the following structure
3     [
4         name,
5         avs,
```

```

6         { [drug]: dosage }
7     ]
8     """
9     name = input("Patient name: ").strip()
10    avs = input("Patient AVS number: ").strip()
11    drugs = {}
12    print("Enter drugs and dosages. Leave drug name empty to finish.")
13    while True:
14        drug = input("Drug name: ").strip()
15        if not drug:
16            break
17        dosage = input("Dosage (string): ").strip()
18        drugs[drug] = dosage
19    return json.dumps([name, avs, drugs]).encode("utf-8")
20
21
22 def build_patient_record() → bytes:
23     """Build a json array with the following structure
24     [
25         name,
26         avs,
27         { [symptom]: temporality }
28     ]
29     """
30    name = input("Patient name: ").strip()
31    avs = input("Patient AVS number: ").strip()
32    symptoms = {}
33    print("Enter symptoms and their temporality. Leave symptom name empty to finish.")
34    while True:
35        sym = input("Symptom: ").strip()
36        if not sym:
37            break
38        temp = input("Temporality (string): ").strip()
39        symptoms[sym] = temp
40    return json.dumps([name, avs, symptoms]).encode("utf-8")

```

Both of these functions give us JSON with the following type:

```

1 type Record = [
2     string,
3     string,
4     {
5         [key: string]: string
6     }
7 ]

```

Before saving, the only way to distinguish between the two types of records is the flag `isPatient`. Once saved to disk, the distinction is whether it comes from `medical_records.txt` or `prescriptions_records.txt`. (The latter isn't that big of an issue since it would be the same if we saved to a database using different tables.) The issue is that if the pharmacy is given patient record with symptoms such as `paracetamol` or `concerta` and the corresponding signature they aren't able to tell that it cannot be used as a prescription.

Furthermore, when looking at what's signed and what's saved, we have another issue.

```
1  def main():
2      #...
3      signature = sign_message(priv, record)
4      save_record_to_db(record, signature, isPatient)
5
6  def save_record_to_db(record: bytes, signature: Signature, isPatient: bool):
7      """Save a patient record to the database accompanied by its signature"""
8      # WARN: There is no guarantee that given signature is for the given record
9      # but this shouldn't be an issue when it comes to saving
10     # WARN: The timestamps isn't part of the signature, which means that it
11     # shouldn't be interpreted as valid because the signature is
12     entry = {
13         "timestamp": datetime.now().isoformat(),
14         "record": record.decode(),
15         "signature": {"r": hex(signature[0]), "s": hex(signature[1])},
16     }
17     # Choose which file to use based on the isPatient flag
18     filename = "medical_records.txt" if isPatient else
19     "prescriptions_records.txt"
20     # WARN: The entry is appended to the file. This will result in an invalid
21     # json structure which will need to be handled correctly when reading
22     # from it
23     with open(filename, "a", encoding="utf-8") as f:
24         f.write(json.dumps(entry, ensure_ascii=False) + "\n")
```

The attached timestamp isn't part of the signature. This means that changing the timestamp wouldn't invalidate a prescription. You could reuse a prescription and simply change the date.

Also related to the idea of having a timestamp, we need to rotate the private key periodically. This rotation shouldn't invalidate previous signatures but currently there is no way to know which public key should be used to validate a signature.

Here's a list of what an attacker could do depending on what they have access to:

- If an attacker can force or accelerate a key rotation, they can invalidate all the previous signatures.
- If an attacker can create a patient record, get it signed and use it at a pharmacy as if it's a prescription it would pass as valid.
- If an attacker had a valid prescription in the past and are able to manipulate the timestamp, they can fake a new identical prescription.

Without an attacker, the key rotation will still invalidate previous signatures.

Once again, this should be fixed because it's a liability issue if prescriptions can be forged using our system or if patients are unable to use their prescriptions after a key rotation.

The first fix is to simply include the timestamp and type of record in the signature

```
1  T = TypeVar("T")
2
3
4  @dataclass(frozen=True)
5  class Record(Generic[T]):
6      """Class representing a value that should be serializable and
7      deserializable
8      and include a timestamp that match its creation.
9      """
10     data: T
11     timestamp: datetime = datetime.now(timezone.utc)
12     pass
13
14     @property
15     def type(self) → str:
16         return self.data.__class__.__name__
17
18     def to_dict(self) → dict:
19         return {
20             "type": self.type,
21             "data": self.data,
22             "timestamp": self.timestamp.isoformat(),
23         }
24
25
26 class DoctorData(NamedTuple):
27     name: str
28     avs: str
29     drugs: dict[str, str]
30
31
32 class PatientData(NamedTuple):
33     name: str
34     avs: str
35     symptoms: dict[str, str]
36
37
38 def build_doctor_record() → Record[DoctorData]:
39     """Build a doctor record"""
40     name = input("Patient name: ").strip()
41     avs = input("Patient AVS number: ").strip()
```

```

42     drugs: dict[str, str] = {}
43     print("Enter drugs and dosages. Leave drug name empty to finish.")
44     while True:
45         drug = input("Drug name: ").strip()
46         if not drug:
47             break
48         dosage = input("Dosage (string): ").strip()
49         drugs[drug] = dosage
50     return Record(data=DoctorData(name, avs, drugs))
51
52 def build_patient_record() → Record[PatientData]:
53     """Build a patient record"""
54     name = input("Patient name: ").strip()
55     avs = input("Patient AVS number: ").strip()
56     symptoms: dict[str, str] = {}
57     print("Enter symptoms and their temporality. Leave symptom name empty to finish.")
58     while True:
59         sym = input("Symptom: ").strip()
60         if not sym:
61             break
62         temp = input("Temporality (string): ").strip()
63         symptoms[sym] = temp
64     return Record(data=PatientData(name, avs, symptoms))
65
66 def save_record_to_db(record: Record, priv: PrivKey):
67     """Save a patient record to the database accompanied by its signature"""
68     json_record = json.dumps(record.to_dict())
69     signature = sign_message(priv, json_record.encode("utf-8"))
70     entry = {
71         "record": json_record,
72         "signature": {"r": hex(signature[0]), "s": hex(signature[1])},
73     }
74     filename: str
75     match record.data:
76         case PatientData():
77             filename = "medical_records.txt"
78         case DoctorData():
79             filename = "prescriptions_records.txt"
80         case _:
81             raise ValueError(f"Unexpected record received {record.type}")
82     # WARN: The entry is appended to the file. This will result in an invalid
83     # json structure which will need to be handled correctly when reading
84     # from it
85     with open(filename, "a", encoding="utf-8") as f:
86         f.write(json.dumps(entry, ensure_ascii=False) + "\n")

```

```

87
88 def main():
89     # ...
90     choice = input("Choose 1 or 2: ").strip()
91     if choice == "1":
92         record = build_doctor_record()
93     elif choice == "2":
94         record = build_patient_record()
95     # NOTE: redundant
96     else:
97         print("Invalid choice")
98         return
99     save_record_to_db(record, priv)

```

This approach fixes the issue of the type of record being lost by including it in the type and serialization. The type and timestamps are both included in the signature.

i Note

This doesn't address the issue of key rotation but helps. Since the timestamp is included in the signature, we need a versioning system for the public keys and the function that would deserialize the record should check the timestamp, find the closest public key before that timestamp.

This isn't implemented in the fix because the original program doesn't implement anything past saving to disk and doing so would take more time without being too useful for this lab.

(The versioning of public keys would also be complex to do right. We probably need a master key to sign the public keys and make sure that those keys are valid when loading the program)

3. Conclusion

Other than cryptographic issues, there are also a lot of problems with the structure of the code. If the goal is to provide a library to use within another program, it should be clearly structured in modules and ensure that the library is used correctly.

We would need classes to ensure that public and private keys are valid on instantiation as well as clear exceptions to handle errors.

Note

The `verify_signature` function is never called. This probably isn't an issue since this is supposed to be a library and we don't have a function that reads a record from the disk.

TODO

Add mentions on AI usage